

Beat by Bit

Jogo de Ritmo em FPGA — Placa DE0-CV

Documentação de Incrementos

Semestre 2 — Revisão de Código

Versão base:	labdig_files/projeto_sem2
Versão atual:	labdig_files_midiAudio/projeto_sem2
Clock do sistema:	50 MHz (Cyclone V)
Taxa de quadros:	60 fps
Baud UART:	115 200 bps

Resumo das mudanças principais

ROM do chart:	4 428 → 9 861 entradas ($\approx 73\text{ s} \rightarrow \approx 164\text{ s}$)
Frame counter:	13 bits → 14 bits
Largura da ROM:	4 bits → 8 bits por entrada
Pacote UART:	20 bytes → 22 bytes
FSM PAUSE:	nova transição PAUSE → IDLE via <code>song_ok</code>
Debounce btn0:	reset desacoplado de <code>limpaR</code>

Sumário

1	Visão Geral do Projeto	2
1.1	Estrutura modular	2
2	Módulo <code>fluxo_dados.v</code> — Expansão da ROM e Telemetria	2
2.1	Expansão da ROM de chart	2
2.1.1	Profundidade da ROM	2
2.1.2	Largura das entradas	3
2.2	Expansão do frame counter	3
2.3	Nova porta de saída: <code>frame_counter_out</code>	4
2.4	Correção no debounce do botão 0	4
2.4.1	Motivação	4
3	Módulo <code>unidade_controle.v</code> — Nova Transição no PAUSE	5
3.1	Descrição dos estados	5
3.2	Nova transição: PAUSE → IDLE	5
3.2.1	Diagrama de estados atualizado	5
3.2.2	Impacto na experiência do usuário	5
3.3	Saídas da UC por estado	5
4	Módulo <code>packet_sender.v</code> — Telemetria de Posição via UART	5
4.1	Nova porta de entrada	6
4.2	Formato do pacote UART	6
4.3	Alterações no buffer e na condição de término	6
4.4	Latência do pacote	8
4.5	Decodificação do frame counter no host	8
5	Módulo de Topo <code>projecto.v</code> — Interligação	8
5.1	Novo fio de interligação	8
5.2	Atualização nas instâncias	8
6	Arquivo de Constraints <code>beat_by_bit.sdc</code>	9
7	Módulos Sem Alteração	9
8	Tabela Consolidada de Mudanças	9
9	Impacto nas Capacidades do Sistema	10

1. Visão Geral do Projeto

O **Beat by Bit** é um jogo de ritmo implementado inteiramente em hardware descritivo Verilog, sintetizado para a FPGA Cyclone V da placa Altera DE0-CV. A lógica do jogo — geração de quadros, leitura do chart, detecção de acertos e erros, contagem de placar — reside nos módulos RTL. Uma aplicação Python em PC recebe o estado do jogo quadro a quadro via UART e realiza toda a renderização gráfica e reprodução de áudio.

1.1. Estrutura modular

A hierarquia de módulos é apresentada na Tabela 1.

Tabela 1: Hierarquia de módulos do projeto

Módulo	Arquivo	Função
beat_by_bit_de0_cv	wrapper board	Mapeia pinos físicos da DE0-CV
proyecto	topo lógico	Interliga todos os submódulos
fluxo_dados	FD	Toda a lógica de jogo
unidade_controle	UC / FSM	Máquina de estados do jogo
packet_sender	PS	Serialização UART por quadro
note_track	NT (×4)	Gerência de notas por trilha
uart_tx	TX	Transmissor UART 8N1
debounce_pulse	DB	Debounce com saída em pulso
contador_m	CM	Contador paramétrico com fim

2. Módulo `fluxo_dados.v` — Expansão da ROM e Telemetria

O módulo `fluxo_dados.v` é o núcleo do jogo. Ele concentra o gerador de quadros, a ROM de chart, o contador de frames, os quatro `note_track` e os contadores de placar. Esta versão recebe duas alterações estruturais significativas: a expansão da memória de chart e a exposição do frame counter para telemetria.

2.1. Expansão da ROM de chart

2.1.1. Profundidade da ROM

A constante `ROM_DEPTH` foi aumentada de **4428** para **9861** entradas. Cada entrada corresponde a um quadro de jogo a 60 fps, portanto:

$$t_{\max} = \frac{9861}{60} \approx 164,4\text{s} \quad (\text{anteriormente } \frac{4428}{60} \approx 73,8\text{s}) \quad (1)$$

Isso permite que o chart cubra a duração integral do arquivo WAV `concerning_hobbits.wav` sem truncamento.

```

1 // Versao base
2 localparam integer ROM_DEPTH = 4428;
3
4 // Versao atual
5 localparam integer ROM_DEPTH = 9861;
```

Listing 1: Parâmetro ROM_DEPTH — antes e depois

2.1.2. Largura das entradas

Simultaneamente, a largura dos registros da ROM foi ampliada de 4 para 8 bits:

```

1 // Versao base
2 reg [3:0] chart_rom [0:ROM_DEPTH-1];
3
4 // Versao atual
5 reg [7:0] chart_rom [0:ROM_DEPTH-1];

```

Listing 2: Tipo do array da ROM

Os bits [3:0] continuam codificando o sinal de spawn para cada uma das quatro trilhas. Os bits [7:4] estão reservados para extensões futuras (ex.: notas com intensidade, notas longas).

2.2. Expansão do frame counter

Com ROM_DEPTH = 9 861, o contador de frames precisa representar valores até 9 860, o que exige no mínimo $\lceil \log_2 9861 \rceil = 14$ bits ($2^{13} = 8192 < 9861 \leq 16383 = 2^{14}$). O registrador foi ampliado de 13 para 14 bits:

```

1 // Versao base
2 reg [12:0] frame_counter; // max 8191
3
4 // Versao atual
5 reg [13:0] frame_counter; // max 16383 > 9861

```

Listing 3: Declaração do frame_counter

A lógica de incremento e o teste de fim de chart (fim_tempo) permanecem funcionalmente idênticos; apenas os literais de inicialização foram atualizados para o novo tamanho:

```

1 // Versao base - reset assíncrono e síncrono agrupados no mesmo bloco
2 always @(posedge clock or posedge reset) begin
3     if (reset || zera_timer) begin
4         frame_counter <= 13'd0;
5     end else if (frame_tick && game_active && frame_counter < ROM_DEPTH -
6         1) begin
7         frame_counter <= frame_counter + 13'd1;
8     end
9 end
10
11 // Versao atual - reset síncrono separado; tamanho 14 bits
12 always @(posedge clock or posedge reset) begin
13     if (reset) begin
14         frame_counter <= 14'd0;
15     end else if (zera_timer) begin
16         frame_counter <= 14'd0;
17     end else if (frame_tick && game_active && frame_counter < ROM_DEPTH -
18         1) begin
19         frame_counter <= frame_counter + 14'd1;
20     end
21 end

```

```
19 end
```

Listing 4: Reset e incremento do frame_counter

Nota sobre o reset: na versão base, `reset` e `zera_timer` compartilhavam o mesmo branch do `if`. Na versão atual eles foram separados em branches distintos, o que é equivalente funcionalmente mas gera um código de prioridade mais explícito e facilita a síntese correta.

2.3. Nova porta de saída: frame_counter_out

Para permitir que o módulo de topo repasse a posição do chart ao `packet_sender`, foi adicionada uma porta de saída de 14 bits:

```
1 // Versao base - apenas frame_tick_out
2 output wire      frame_tick_out
3
4 // Versao atual - frame_tick_out + frame_counter_out
5 output wire      frame_tick_out,
6 output wire [13:0] frame_counter_out
```

Listing 5: Nova porta de saída em fluxo_dados

A atribuição é trivial:

```
1 assign frame_tick_out      = frame_tick;
2 assign frame_counter_out = frame_counter; // linha adicionada
```

2.4. Correção no debounce do botão 0

O debounce do botão 0 (`song_ok_pulso`) foi desacoplado do sinal `limpaR` no pino de reset do submódulo `debounce_pulse`:

```
1 // Versao base
2 debounce_pulse #(.DEBOUNCE_CYCLES(BTN_DEBOUNCE_CYCLES))
3     u_db_btn0 (.clock(clock), .reset(reset | limpaR), ...);
4
5 // Versao atual
6 debounce_pulse #(.DEBOUNCE_CYCLES(BTN_DEBOUNCE_CYCLES))
7     u_db_btn0 (.clock(clock), .reset(reset), ...);
```

Listing 6: Reset do debounce — botão 0

2.4.1. Motivação

A UC mantém `limpaR` em nível alto nos estados `idle` e `select`. Na versão base, isso mantinha o debounce do botão 0 continuamente resetado enquanto o jogo estava em `select`, tornando impossível detectar o pressionamento de seleção de música nesse estado. Ao remover `limpaR` da condição de reset, o debounce opera normalmente durante todo o ciclo — o que é seguro porque `song_ok_pulso` só é interpretado pela UC nos estados `select` e `pause`.

3. Módulo `unidade_controle.v` — Nova Transição no PAUSE

A Unidade de Controle (UC) implementa a FSM principal do jogo em Verilog combinacional + registrador de estado. A versão atual introduz uma nova transição de saída do estado `pause`.

3.1. Descrição dos estados

Tabela 2: Estados da FSM

Código	Estado	Descrição
0000	<code>idle</code>	Tela inicial; contadores e placar limpos
0001	<code>countdown</code>	Contagem regressiva de 3 segundos
0010	<code>play</code>	Jogo ativo; <code>game_active = 1</code>
0011	<code>pause</code>	Jogo pausado
0100	<code>end_win</code>	Vitória (chart concluído sem 10 erros)
0101	<code>end_lose</code>	Derrota (10 erros acumulados)
0110	<code>select</code>	Seleção de música

3.2. Nova transição: PAUSE → IDLE

Na versão base, o único caminho de saída do estado `pause` era retornar ao `play` via `start`. Na versão atual, foi adicionada uma transição que retorna ao `idle` quando `song_ok` (botão 0) é pressionado:

```

1 // Versao base
2 pause: Eprox = start ? play : pause;
3
4 // Versao atual
5 pause: Eprox = song_ok ? idle :
6               start  ? play  : pause;
```

Listing 7: Lógica de próximo estado — estado `pause`

3.2.1. Diagrama de estados atualizado

3.2.2. Impacto na experiência do usuário

Antes da mudança, abandonar uma música pausada exigia pressionar o botão de reset físico da placa. Agora, pressionar o botão 0 (`song_ok`) em `pause` retorna ao `idle`, de onde o fluxo segue normalmente: `idle` → `select` → `countdown` → `play`. O placar é zerado automaticamente porque os estados `idle` e `select` mantêm `limpaR = 1`.

3.3. Saídas da UC por estado

4. Módulo `packet_sender.v` — Telemetria de Posição via UART

O `packet_sender.v` monta e envia um pacote de bytes pela UART a cada `frame_tick` (≈ 60 pacotes/segundo). A versão atual adiciona o frame counter ao pacote, expandindo-o de 20 para 22 bytes.

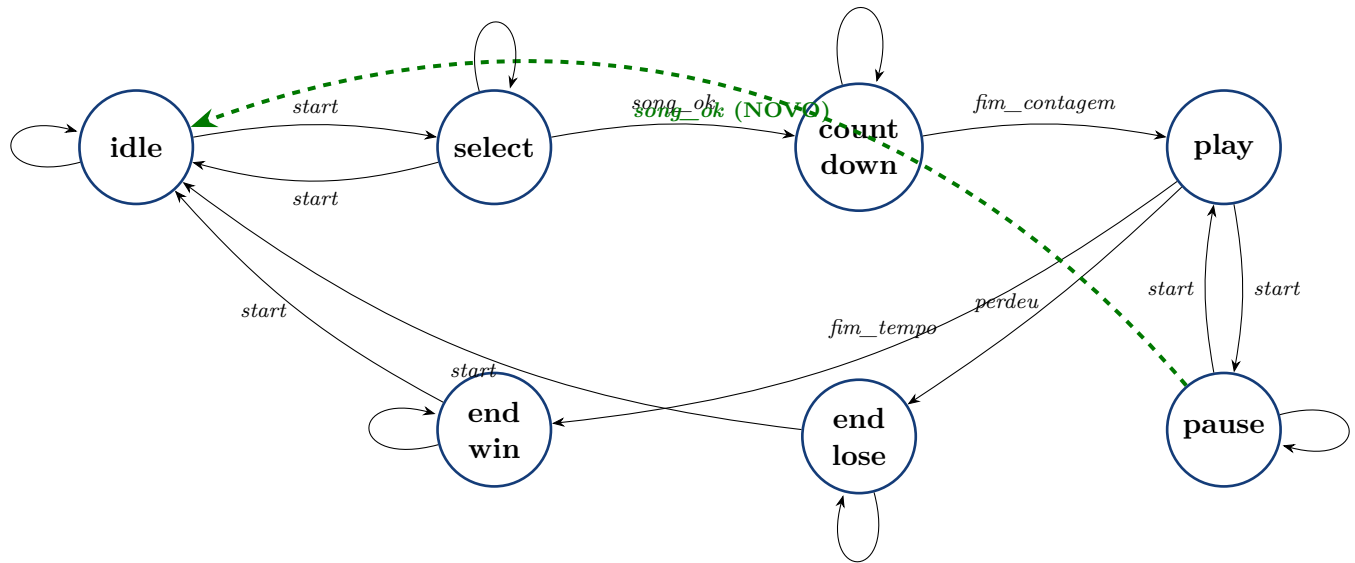


Figura 1: FSM do jogo — transição nova destacada em verde tracejado

Tabela 3: Tabela de saídas da UC por estado

Estado	limpaR	zera_timer	conta_timer	game_active	registraR
idle	1	1	0	0	0
select	1	1	0	0	0
countdown	0	0	1	0	0
play	0	0	1	1	jogada
pause	0	0	0	0	0
end_win	0	1	0	0	0
end_lose	0	1	0	0	0

4.1. Nova porta de entrada

```

1 // Versao base - sem frame_counter
2 // Versao atual
3 input wire [13:0] frame_counter, // progresso do chart

```

Listing 8: Declaração de porta adicionada ao packet_sender

4.2. Formato do pacote UART

A Tabela 4 detalha o formato completo dos dois pacotes.

4.3. Alterações no buffer e na condição de término

```

1 // Versao base
2 reg [7:0] pkt [0:19]; // 20 bytes
3
4 // Versao atual
5 reg [7:0] pkt [0:21]; // 22 bytes

```

Listing 9: Buffer do pacote

Tabela 4: Formato do pacote UART — comparação entre versões

Byte	Campo	Valor / Sinal	Base	Atual
[0]	Marcador início	0x55	✓	✓
[1]	Estado do jogo	{4'b0, game_state}	✓	✓
[2]	Countdown	countdown_sec	✓	✓
[3]	T0 nota 0	t0n0 (age; 0xFF=vazio)	✓	✓
[4]	T0 nota 1	t0n1	✓	✓
[5]	T0 nota 2	t0n2	✓	✓
[6]	T1 nota 0	t1n0	✓	✓
[7]	T1 nota 1	t1n1	✓	✓
[8]	T1 nota 2	t1n2	✓	✓
[9]	T2 nota 0	t2n0	✓	✓
[10]	T2 nota 1	t2n1	✓	✓
[11]	T2 nota 2	t2n2	✓	✓
[12]	T3 nota 0	t3n0	✓	✓
[13]	T3 nota 1	t3n1	✓	✓
[14]	T3 nota 2	t3n2	✓	✓
[15]	Score alto	score[15:8]	✓	✓
[16]	Score baixo	score[7:0]	✓	✓
[17]	Misses	misses	✓	✓
[18]	Combo	combo	✓	✓
[19]	Frame alto	{2'b00, frame_counter[13:8]}	0xAA*	✓
[20]	Frame baixo	frame_counter[7:0]	—	✓
[21]	Marcador fim	0xAA	—	✓
Total			20 B	22 B

* Na versão base, o marcador 0xAA ocupava o byte [19].

```

1 // Versao base
2 pkt[19] <= 8'hAA;
3
4 // Versao atual
5 pkt[19] <= {2'b00, frame_counter[13:8]};
6 pkt[20] <= frame_counter[7:0];
7 pkt[21] <= 8'hAA;

```

Listing 10: Preenchimento do pacote no estado WAIT

```

1 // Versao base
2 if (byte_idx == 19) begin
3     state <= DONE;
4 end
5
6 // Versao atual
7 if (byte_idx == 21) begin
8     state <= DONE;
9 end

```

Listing 11: Condição de último byte no estado SEND

4.4. Latência do pacote

Com 22 bytes a 115 200 bps (10 bits por byte em modo 8N1):

$$t_{\text{pacote}} = \frac{22 \times 10}{115200} \approx 1,91 \text{ ms} \quad \left(\text{período de quadro} = \frac{1}{60} \approx 16,67 \text{ ms} \right) \quad (2)$$

O tempo de transmissão representa apenas 11,4% do período de quadro, não havendo risco de o `packet_sender` não terminar antes do próximo `frame_tick`.

4.5. Decodificação do frame counter no host

O Python pode reconstruir a posição do chart a partir dos dois bytes transmitidos com a seguinte operação:

$$\text{frame} = (\text{pkt}[19] \& 0x3F) \ll 8 \mid \text{pkt}[20] \quad (3)$$

Isso permite sincronizar o início da reprodução de áudio com o frame exato em que o chart se encontra, eliminando desalinhamentos acumulados por variações de clock entre o PC e a FPGA.

5. Módulo de Topo `projecto.v` — Interligação

O módulo de topo conecta `fluxo_dados`, `unidade_controle` e `packet_sender`. As alterações aqui são puramente de cabeamento, decorrentes das novas portas adicionadas nos módulos internos.

5.1. Novo fio de interligação

```
1 // Versao base - nao existe
2 // Versao atual
3 wire [13:0] s_frame_counter;
```

Listing 12: Declaração do fio `s_frame_counter`

5.2. Atualização nas instâncias

```
1 fluxo_dados u_fd (
2     ...
3     .frame_tick_out    (s_frame_tick),
4     .frame_counter_out (s_frame_counter)    // NOVO
5 );
```

Listing 13: Instância de `fluxo_dados` — nova conexão

```
1 packet_sender u_ps (
2     ...
3     .combo            (s_combo),
4     .frame_counter    (s_frame_counter),    // NOVO
5     .uart_tx_out      (uart_tx_out)
6 );
```

Listing 14: Instância de `packet_sender` — nova conexão

6. Arquivo de Constraints beat_by_bit.sdc

O único ponto alterado no arquivo de timing constraints é a referência ao pino de clock:

```
1 # Versao base
2 create_clock -name clk -period 20.000 [get_ports {CLOCK_50}]
3
4 # Versao atual
5 create_clock -name clk -period 20.000 [get_ports {clock}]
```

Listing 15: Constraint de clock

A mudança reflete a renomeação interna do pino no módulo wrapper `beat_by_bit_de0_cv.v`, que mapeia o sinal `CLOCK_50` da placa para o nome genérico `clock`. O período de 20 ns corresponde ao clock de 50 MHz e permanece inalterado.

7. Módulos Sem Alteração

Os módulos listados abaixo permaneceram funcionalmente idênticos entre as duas versões. Nenhuma porta, parâmetro ou lógica interna foi modificada.

Tabela 5: Módulos inalterados

Arquivo	Função
<code>uart_tx.v</code>	Transmissor UART 8N1 paramétrico
<code>note_track.v</code>	Gerência de notas por trilha (até 3 simultâneas)
<code>edge_detector.v</code>	Detector de borda de subida/descida
<code>debounce_pulse.v</code>	Debounce com saída em pulso
<code>hexa7seg.v</code>	Decodificador para display 7 segmentos
<code>lfsr8.v</code>	LFSR de 8 bits
<code>registrador_4.v</code>	Registrador de 4 bits com enable e clear
<code>contador_m.v</code>	Contador paramétrico M com saída de fim
<code>beat_by_bit_de0_cv.v</code>	Wrapper de pinos da placa DE0-CV

8. Tabela Consolidada de Mudanças

Tabela 6: Resumo consolidado de todas as alterações

Arquivo	Versão base	Versão atual
<code>fluxo_dados.v</code>	ROM_DEPTH = 4428	ROM_DEPTH = 9861
<code>fluxo_dados.v</code>	reg [3:0] chart_rom	reg [7:0] chart_rom
<code>fluxo_dados.v</code>	reg [12:0] frame_counter	reg [13:0] frame_counter
<code>fluxo_dados.v</code>	Sem porta frame_counter_out	output wire [13:0] frame_counter_out

Arquivo	Versão base	Versão atual
fluxo_dados.v	debounce btn0: .reset(reset limpaR)	debounce btn0: .reset(reset)
unidade_controle.v	pause: start → play	pause: song_ok → idle, start → play
packet_sender.v	Sem porta frame_counter	input wire [13:0] frame_counter
packet_sender.v	pkt[0:19] (20 B)	pkt[0:21] (22 B)
packet_sender.v	pkt[19] = 0xAA	pkt[19..20] = frame_counter pkt[21] = 0xAA
packet_sender.v	byte_idx == 19	byte_idx == 21
proyecto.v	Sem s_frame_counter	wire [13:0] s_frame_counter
beat_by_bit.sdc	get_ports {CLOCK_50}	get_ports {clock}

9. Impacto nas Capacidades do Sistema

Tabela 7: Comparativo de capacidades

Capacidade	Versão base	Versão atual
Duração máxima do chart	≈73 s	≈164 s
Bits do frame counter	13 bits	14 bits
Largura da ROM	4 bits/frame	8 bits/frame
Tamanho do pacote UART	20 bytes	22 bytes
Tempo de tx por pacote	≈1,74 ms	≈1,91 ms
Telemetria de posição	Não	Sim
Saída do PAUSE sem reset	Não	Sim (via botão 0)

Compatibilidade de interface: a interface de I/O externa do sistema (pinos físicos da DE0-CV, protocolo UART de nível físico) permanece idêntica. As mudanças são internas ao projeto RTL e ao protocolo de dados do pacote. O software Python precisa ser atualizado para ler 22 bytes por pacote (em vez de 20) e para decodificar os bytes [19] e [20] como `frame_counter`.